Nithin Gowtham Saravanan

CREATING A CUSTOM HARDWARE IP AND INTERFACING IT WITH SOFTWARE

TABLE OF CONTENTS

Nithin Gowtham Saravanan

Introduction

The purpose of this laboratory exercise was to design and integrate a custom AXI4-Lite hardware peripheral into a Zynq Processing System on the Zybo Z7-10 board. Unlike the previous lab, which used predefined IP blocks and a soft processor, this lab required creating a custom multiplier IP in programmable logic and interfacing it with the ARM Cortex-A9 processor in the processing system. The ARM processor writes operands to the custom hardware registers and reads back the computed result through AXI communication. This lab highlights the importance of hardware/software co-design and demonstrates how custom accelerators can be implemented in FPGA fabric and controlled through embedded software in modern embedded systems.

Procedure

1. Zynq Processing System Setup

1. A working directory was created in the home folder to organize all files related to Laboratory Exercise #3.
2. The Vivado environment variables were loaded on the workstation using the appropriate setup command, and Vivado 2022.1 was launched.
3. A new project was created by selecting the **Boards** option and choosing the **Zybo Z7-10** board.
4. A new block design was created and named *multiply*.
5. The **ZYNQ7 Processing System (PS)** IP was added to the block diagram.
6. The ZYBO_Z7_B2_.tcl preset file was downloaded and imported through the **Re-customize IP** window.
7. All peripheral I/O pins were unchecked, and only **UART1** was enabled to allow serial communication between the Zybo Z7-10 board and the workstation.

2. Creation and Packaging of Custom Multiply IP

8. From the Vivado Tools menu, **Create and Package New IP** was selected.
9. A new AXI4-Lite slave peripheral named *multiply* was created with:
    - AXI4-Lite interface
    - Slave mode
    - 32-bit data width
    - Four registers
10. The IP was generated, and the **Edit IP** option was selected to open the custom IP project in a new Vivado window.
11. The file multiply_v1_0_S00_AXI.v was opened.
12. The code responsible for writing to slv_reg2 was commented out to disable software write access to the result register.
13. The multiplication logic provided in the lab manual was inserted so that the product of slv_reg0 and slv_reg1 was assigned to slv_reg2 inside the AXI clocked process.

14. After verifying the modifications, the IP was re-packaged using **Review and Package → Re-package IP**.

The modified user logic is shown in **APPENDIX A**.

3. Integration of Multiply IP into PS System

15. The Vivado window containing the Zynq PS system was selected, and the newly created *multiply* IP was added to the block diagram.
16. Connection Automation was executed to automatically connect the AXI interface and clock signals. The **Regenerate Layout** option was selected to organize the design.
17. The design was validated using **Validate Design** to ensure there were no connection errors.
18. An HDL wrapper was created, and the bitstream was generated.
19. After successful bitstream generation, the hardware design was exported as an .xsa file including the bitstream.

The final block diagram is provided in **APPENDIX B**.

4. Software Development and Testing in Vitis

20. Vitis IDE was launched, and a new **Application Project** was created.
21. In the Platform page, the exported hardware design file was selected using the **Create a new hardware platform** option.
22. An application name was provided, and the **Hello World** template was selected.
23. The helloworld.c file was modified to:

- Write values to slv_reg0
- Write values to slv_reg1
- Read the multiplication result from slv_reg2
- Print the operands and result using printf

The source code is included in **APPENDIX C**.

24. The hardware platform project and software system project were built successfully, generating the application binary file (.elf).
25. The FPGA board was programmed using the **Program Device** option.
26. The serial console was launched using picocom with the command specified in the lab manual to view the printf outputs.

The terminal launch command is shown in **APPENDIX D**, and the multiplication results observed on the serial console are shown in **APPENDIX E**.

(The correct operation of the system was verified and demonstrated to the TA.)

Results

The custom multiplier IP operated correctly on the Zybo Z7-10 board. When values were written to slv_reg0 and slv_reg1 from the ARM processor, the multiplication was performed in programmable logic, and the result was stored in slv_reg2. The ARM processor successfully read the result and printed it to the serial console using UART1.

The picocom output confirmed that the multiplication results matched expected arithmetic values for various test cases. This verified correct AXI register mapping, successful PS-to-PL communication, and proper functionality of the custom hardware logic.

The console output demonstrating correct multiplication results is shown in **APPENDIX E (picocom Output)**.

The system functioned correctly without requiring debugging modifications. The integration of custom hardware logic with processor-controlled software demonstrated proper hardware/software interaction and AXI communication. The final working system was successfully demonstrated to the TA.

Conclusion

In conclusion, the third laboratory exercise provided us with hands-on experience in creating and integrating custom IP modules, which are essential for Zynq Processing System-based systems. By developing a custom peripheral for integer multiplication and integrating it into a microprocessor system, we gained practical insight into hardware/software co-design and the interaction between programmable logic and the processing system. This exercise not only equipped us with valuable skills for building specialized embedded systems but also strengthened our understanding of AXI-based communication and custom peripheral development. It emphasized the importance of tailored hardware solutions for improving performance and efficiency in various industries. Moving forward, this knowledge will serve us well in tackling real-world engineering challenges with confidence and expertise, particularly in the design of optimized and scalable embedded FPGA systems.

Nithin Gowtham Saravanan

Knowledge Transfer

a) Recall that 'slv reg0', 'slv reg1', and 'slv reg2' are all 32-bit registers. What values of 'slv reg0' and 'slv reg1' would produce incorrect results from the multiplication block? What is the name commonly assigned to this type of computation error, and how would you correct this? Provide a Verilog example and explain what you would change during the creation of the corrected peripheral.

Since slv_reg0, slv_reg1, and slv_reg2 are 32-bit registers, multiplying two large 32-bit values can produce a 64-bit result. If the product exceeds 32 bits, the upper bits are truncated when stored in slv_reg2, resulting in incorrect output. This type of error is called **overflow**.

To correct this, the multiplier result width must be increased. For example:

```
reg [63:0] product;
always @(posedge S_AXI_ACLK) begin
    if (!S_AXI_ARESETN)
        product <= 64'd0;
    else
        product <= slv_reg0 * slv_reg1;
end
```

The 64-bit result can then be split across two 32-bit registers (e.g., slv_reg2 and slv_reg3) or the AXI data width can be modified. Updating the register width during peripheral creation prevents loss of significant bits.

b) In this exercise, we wrote the multiplier and the multiplicand to the input registers, followed by a read from the output register for the multiplication result. Is it possible that we end up reading the output register before the correct result is available? Why?

Yes, there is a possibility of reading the output register before the correct result is available. If the multiplication operation required multiple clock cycles to complete, and the processor performed a read operation immediately after writing the inputs, the result register might still contain a previous or incomplete value.

In this lab implementation, the multiplication is performed inside a clocked always block and completes within one clock cycle, meaning the result becomes available on the next clock edge. However, in more complex designs where multiplication is pipelined or implemented using multi-cycle arithmetic units, additional synchronization logic such as a "valid" or "ready" signal would be necessary to ensure the processor reads the result only after computation has completed.

c) While creating the multiply IP, we kept the interface mode as ”Slave”. Suppose we change this mode to ”Master”, do you think it will impact our experiment? Why?

The AXI interface mode determines whether the peripheral initiates transactions or responds to them. In slave mode, the peripheral waits for read and write requests from the processor and responds accordingly. This is appropriate in our design because the ARM processor writes operands to the multiplier registers and reads back the result.

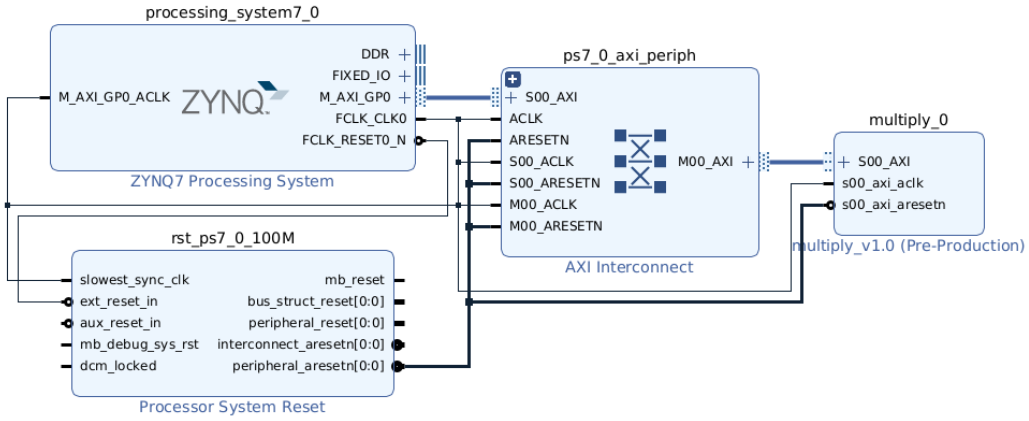
If the interface were changed to master mode, the multiplier IP would attempt to initiate AXI transactions on its own. This would significantly alter the system behavior and require additional logic to control memory accesses. Since our experiment relies on the processor controlling the peripheral, changing the mode to master would prevent the intended read/write interaction and would fundamentally change the architecture of the design.

Appendix

APPENDIX A (user logic in multiply_v1_0_S00_AXI.v)

```
// Add user logic here
reg [0 : C_S_AXI_DATA_WIDTH-1] tmp_reg ;    // tmp_reg to store the computed result
always @( posedge S_AXI_ACLK ) begin
    // For a reset, set the output register (slv_reg2) and the temp reg to 0
    if( S_AXI_ARESETN == 1'b0) begin
        slv_reg2 <=0;
        tmp_reg <=0;
    end
    // Multiply operand 1 (slv_reg0) and operand 2 (slv_reg1); Store the result in output register (slv_reg2)
    else begin
        tmp_reg <= slv_reg0 * slv_reg1 ;
        slv_reg2 <= tmp_reg ;
    end
end
// User logic ends
```

APPENDIX B (Block Diagram)



APPENDIX C (Vitis source file)

multiply_test_system | multiply_test | helloworld.c | platform.c | platform.h | xparameters.h | multiply.h | xil_io.h | xil_io.h

```c
/*
 * helloworld.c: simple test application
 *
 * This application configures UART 16550 to baud rate 9600.
 * PS7 UART (Zynq) is not initialized by this application, since
 * bootrom/bsp configures it to baud rate 115200
 *
 * ------------------------------------------------
 * | UART TYPE   BAUD RATE                        |
 * ------------------------------------------------
 *   uartns550   9600
 *   uartlite    Configurable only in HW design
 *   ps7_uart    115200 (configured by bootrom/bsp)
 */

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "multiply.h"
#include "xil_io.h"



int main()
{
    init_platform();    // Initialize the platform

    int reg_0, reg_1, reg_2;

    for( reg_0 = 0; reg_0 <= 16; reg_0++)   // First number for multiplication -> reg_0
    {
        // Write reg_0 with the outer loop values as operand 1
        MULTIPLY_mWriteReg(XPAR_MULTIPLY_0_S00_AXI_BASEADDR, MULTIPLY_S00_AXI_SLV_REG0_OFFSET, reg_0);

        for( reg_1 = 0; reg_1 <= 16; reg_1++)   // Second number for multiplication -> reg_1
        {
            printf("Operand 1 = %d; \t", reg_0 );

            // Write reg_1 with the inner loop values as operand 2
            MULTIPLY_mWriteReg(XPAR_MULTIPLY_0_S00_AXI_BASEADDR, MULTIPLY_S00_AXI_SLV_REG1_OFFSET, reg_1);
            printf("Operand 2 = %d \t -> \t", reg_1 );

            // Store the computed result of multiplication from slv_reg2 in the created "multiply" IP
            reg_2 = MULTIPLY_mReadReg(XPAR_MULTIPLY_0_S00_AXI_BASEADDR, MULTIPLY_S00_AXI_SLV_REG2_OFFSET);
            printf("Multiplied Result = %d \n", reg_2);
        }
    }

    cleanup_platform(); // Default cleanup code
    return 0;
}
```

APPENDIX D (picocom Terminal Launch)

```
nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749$ source /opt/coe/Xilinx/Vivado/2023.1/settings64.sh
nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749$ picocom -b 115200 /dev/ttyUSB1
picocom v3.1

port is        : /dev/ttyUSB1
flowcontrol    : none
baudrate is    : 115200
parity is      : none
databits are   : 8
stopbits are   : 1
escape is      : C-a
local echo is  : no
noinit is      : no
noreset is     : no
hangup is      : no
nolock is      : no
send_cmd is    : sz -vv
receive_cmd is : rz -vv -E
imap is        :
omap is        :
emap is        : crcrlf,delbs,
logfile is     : none
initstring     : none
exit_after is  : not set
exit is        : no

Type [C-a] [C-h] to see available commands
Terminal ready
```

Nithin Gowtham Saravanan

APPENDIX E (picocom Output)

```
nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749$ picocom -b 115200 /dev/ttyUSB1
picocom v3.1

port is        : /dev/ttyUSB1
flowcontrol    : none
baudrate is    : 115200
parity is      : none
databits are   : 8
stopbits are   : 1
escape is      : C-a
local echo is  : no
noinit is      : no
noreset is     : no
hangup is      : no
nolock is      : no
send_cmd is    : sz -vv
receive_cmd is : rz -vv -E
imap is        :
omap is        :
emap is        : crcrlf,delbs,
logfile is     : none
initstring     : none
exit_after is  : not set
exit is        : no

Type [C-a] [C-h] to see available commands
Terminal ready
Operand 1 = 0;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 1    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 2    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 3    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 4    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 5    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 6    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 7    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 8    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 9    ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 10   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 11   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 12   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 13   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 14   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 15   ->     Multiplied Result = 0
Operand 1 = 0;  Operand 2 = 16   ->     Multiplied Result = 0
Operand 1 = 1;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 1;  Operand 2 = 1    ->     Multiplied Result = 1
Operand 1 = 1;  Operand 2 = 2    ->     Multiplied Result = 2
Operand 1 = 1;  Operand 2 = 3    ->     Multiplied Result = 3
Operand 1 = 1;  Operand 2 = 4    ->     Multiplied Result = 4
Operand 1 = 1;  Operand 2 = 5    ->     Multiplied Result = 5
Operand 1 = 1;  Operand 2 = 6    ->     Multiplied Result = 6
Operand 1 = 1;  Operand 2 = 7    ->     Multiplied Result = 7
Operand 1 = 1;  Operand 2 = 8    ->     Multiplied Result = 8
Operand 1 = 1;  Operand 2 = 9    ->     Multiplied Result = 9
Operand 1 = 1;  Operand 2 = 10   ->     Multiplied Result = 10
Operand 1 = 1;  Operand 2 = 11   ->     Multiplied Result = 11
Operand 1 = 1;  Operand 2 = 12   ->     Multiplied Result = 12
Operand 1 = 1;  Operand 2 = 13   ->     Multiplied Result = 13
Operand 1 = 1;  Operand 2 = 14   ->     Multiplied Result = 14
Operand 1 = 1;  Operand 2 = 15   ->     Multiplied Result = 15
Operand 1 = 1;  Operand 2 = 16   ->     Multiplied Result = 16
Operand 1 = 2;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 2;  Operand 2 = 1    ->     Multiplied Result = 2
Operand 1 = 2;  Operand 2 = 2    ->     Multiplied Result = 4
Operand 1 = 2;  Operand 2 = 3    ->     Multiplied Result = 6
Operand 1 = 2;  Operand 2 = 4    ->     Multiplied Result = 8
Operand 1 = 2;  Operand 2 = 5    ->     Multiplied Result = 10
Operand 1 = 2;  Operand 2 = 6    ->     Multiplied Result = 12
Operand 1 = 2;  Operand 2 = 7    ->     Multiplied Result = 14
Operand 1 = 2;  Operand 2 = 8    ->     Multiplied Result = 16
```

```
                                                    nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749
Operand 1 = 2;  Operand 2 = 8    ->     Multiplied Result = 16
Operand 1 = 2;  Operand 2 = 9    ->     Multiplied Result = 18
Operand 1 = 2;  Operand 2 = 10   ->     Multiplied Result = 20
Operand 1 = 2;  Operand 2 = 11   ->     Multiplied Result = 22
Operand 1 = 2;  Operand 2 = 12   ->     Multiplied Result = 24
Operand 1 = 2;  Operand 2 = 13   ->     Multiplied Result = 26
Operand 1 = 2;  Operand 2 = 14   ->     Multiplied Result = 28
Operand 1 = 2;  Operand 2 = 15   ->     Multiplied Result = 30
Operand 1 = 2;  Operand 2 = 16   ->     Multiplied Result = 32
Operand 1 = 3;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 3;  Operand 2 = 1    ->     Multiplied Result = 3
Operand 1 = 3;  Operand 2 = 2    ->     Multiplied Result = 6
Operand 1 = 3;  Operand 2 = 3    ->     Multiplied Result = 9
Operand 1 = 3;  Operand 2 = 4    ->     Multiplied Result = 12
Operand 1 = 3;  Operand 2 = 5    ->     Multiplied Result = 15
Operand 1 = 3;  Operand 2 = 6    ->     Multiplied Result = 18
Operand 1 = 3;  Operand 2 = 7    ->     Multiplied Result = 21
Operand 1 = 3;  Operand 2 = 8    ->     Multiplied Result = 24
Operand 1 = 3;  Operand 2 = 9    ->     Multiplied Result = 27
Operand 1 = 3;  Operand 2 = 10   ->     Multiplied Result = 30
Operand 1 = 3;  Operand 2 = 11   ->     Multiplied Result = 33
Operand 1 = 3;  Operand 2 = 12   ->     Multiplied Result = 36
Operand 1 = 3;  Operand 2 = 13   ->     Multiplied Result = 39
Operand 1 = 3;  Operand 2 = 14   ->     Multiplied Result = 42
Operand 1 = 3;  Operand 2 = 15   ->     Multiplied Result = 45
Operand 1 = 3;  Operand 2 = 16   ->     Multiplied Result = 48
Operand 1 = 4;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 4;  Operand 2 = 1    ->     Multiplied Result = 4
Operand 1 = 4;  Operand 2 = 2    ->     Multiplied Result = 8
Operand 1 = 4;  Operand 2 = 3    ->     Multiplied Result = 12
Operand 1 = 4;  Operand 2 = 4    ->     Multiplied Result = 16
Operand 1 = 4;  Operand 2 = 5    ->     Multiplied Result = 20
Operand 1 = 4;  Operand 2 = 6    ->     Multiplied Result = 24
Operand 1 = 4;  Operand 2 = 7    ->     Multiplied Result = 28
Operand 1 = 4;  Operand 2 = 8    ->     Multiplied Result = 32
Operand 1 = 4;  Operand 2 = 9    ->     Multiplied Result = 36
Operand 1 = 4;  Operand 2 = 10   ->     Multiplied Result = 40
Operand 1 = 4;  Operand 2 = 11   ->     Multiplied Result = 44
Operand 1 = 4;  Operand 2 = 12   ->     Multiplied Result = 48
Operand 1 = 4;  Operand 2 = 13   ->     Multiplied Result = 52
Operand 1 = 4;  Operand 2 = 14   ->     Multiplied Result = 56
Operand 1 = 4;  Operand 2 = 15   ->     Multiplied Result = 60
Operand 1 = 4;  Operand 2 = 16   ->     Multiplied Result = 64
Operand 1 = 5;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 5;  Operand 2 = 1    ->     Multiplied Result = 5
Operand 1 = 5;  Operand 2 = 2    ->     Multiplied Result = 10
Operand 1 = 5;  Operand 2 = 3    ->     Multiplied Result = 15
Operand 1 = 5;  Operand 2 = 4    ->     Multiplied Result = 20
Operand 1 = 5;  Operand 2 = 5    ->     Multiplied Result = 25
Operand 1 = 5;  Operand 2 = 6    ->     Multiplied Result = 30
Operand 1 = 5;  Operand 2 = 7    ->     Multiplied Result = 35
Operand 1 = 5;  Operand 2 = 8    ->     Multiplied Result = 40
Operand 1 = 5;  Operand 2 = 9    ->     Multiplied Result = 45
Operand 1 = 5;  Operand 2 = 10   ->     Multiplied Result = 50
Operand 1 = 5;  Operand 2 = 11   ->     Multiplied Result = 55
Operand 1 = 5;  Operand 2 = 12   ->     Multiplied Result = 60
Operand 1 = 5;  Operand 2 = 13   ->     Multiplied Result = 65
Operand 1 = 5;  Operand 2 = 14   ->     Multiplied Result = 70
Operand 1 = 5;  Operand 2 = 15   ->     Multiplied Result = 75
Operand 1 = 5;  Operand 2 = 16   ->     Multiplied Result = 80
Operand 1 = 6;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 6;  Operand 2 = 1    ->     Multiplied Result = 6
Operand 1 = 6;  Operand 2 = 2    ->     Multiplied Result = 12
Operand 1 = 6;  Operand 2 = 3    ->     Multiplied Result = 18
Operand 1 = 6;  Operand 2 = 4    ->     Multiplied Result = 24
Operand 1 = 6;  Operand 2 = 5    ->     Multiplied Result = 30
Operand 1 = 6;  Operand 2 = 6    ->     Multiplied Result = 36
Operand 1 = 6;  Operand 2 = 7    ->     Multiplied Result = 42
Operand 1 = 6;  Operand 2 = 8    ->     Multiplied Result = 48
Operand 1 = 6;  Operand 2 = 9    ->     Multiplied Result = 54
Operand 1 = 6;  Operand 2 = 10   ->     Multiplied Result = 60
Operand 1 = 6;  Operand 2 = 11   ->     Multiplied Result = 66
Operand 1 = 6;  Operand 2 = 12   ->     Multiplied Result = 72
```

```
                                                    nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749
Operand 1 = 6;  Operand 2 = 12   ->     Multiplied Result = 72
Operand 1 = 6;  Operand 2 = 13   ->     Multiplied Result = 78
Operand 1 = 6;  Operand 2 = 14   ->     Multiplied Result = 84
Operand 1 = 6;  Operand 2 = 15   ->     Multiplied Result = 90
Operand 1 = 6;  Operand 2 = 16   ->     Multiplied Result = 96
Operand 1 = 7;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 7;  Operand 2 = 1    ->     Multiplied Result = 7
Operand 1 = 7;  Operand 2 = 2    ->     Multiplied Result = 14
Operand 1 = 7;  Operand 2 = 3    ->     Multiplied Result = 21
Operand 1 = 7;  Operand 2 = 4    ->     Multiplied Result = 28
Operand 1 = 7;  Operand 2 = 5    ->     Multiplied Result = 35
Operand 1 = 7;  Operand 2 = 6    ->     Multiplied Result = 42
Operand 1 = 7;  Operand 2 = 7    ->     Multiplied Result = 49
Operand 1 = 7;  Operand 2 = 8    ->     Multiplied Result = 56
Operand 1 = 7;  Operand 2 = 9    ->     Multiplied Result = 63
Operand 1 = 7;  Operand 2 = 10   ->     Multiplied Result = 70
Operand 1 = 7;  Operand 2 = 11   ->     Multiplied Result = 77
Operand 1 = 7;  Operand 2 = 12   ->     Multiplied Result = 84
Operand 1 = 7;  Operand 2 = 13   ->     Multiplied Result = 91
Operand 1 = 7;  Operand 2 = 14   ->     Multiplied Result = 98
Operand 1 = 7;  Operand 2 = 15   ->     Multiplied Result = 105
Operand 1 = 7;  Operand 2 = 16   ->     Multiplied Result = 112
Operand 1 = 8;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 8;  Operand 2 = 1    ->     Multiplied Result = 8
Operand 1 = 8;  Operand 2 = 2    ->     Multiplied Result = 16
Operand 1 = 8;  Operand 2 = 3    ->     Multiplied Result = 24
Operand 1 = 8;  Operand 2 = 4    ->     Multiplied Result = 32
Operand 1 = 8;  Operand 2 = 5    ->     Multiplied Result = 40
Operand 1 = 8;  Operand 2 = 6    ->     Multiplied Result = 48
Operand 1 = 8;  Operand 2 = 7    ->     Multiplied Result = 56
Operand 1 = 8;  Operand 2 = 8    ->     Multiplied Result = 64
Operand 1 = 8;  Operand 2 = 9    ->     Multiplied Result = 72
Operand 1 = 8;  Operand 2 = 10   ->     Multiplied Result = 80
Operand 1 = 8;  Operand 2 = 11   ->     Multiplied Result = 88
Operand 1 = 8;  Operand 2 = 12   ->     Multiplied Result = 96
Operand 1 = 8;  Operand 2 = 13   ->     Multiplied Result = 104
Operand 1 = 8;  Operand 2 = 14   ->     Multiplied Result = 112
Operand 1 = 8;  Operand 2 = 15   ->     Multiplied Result = 120
Operand 1 = 8;  Operand 2 = 16   ->     Multiplied Result = 128
Operand 1 = 9;  Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 9;  Operand 2 = 1    ->     Multiplied Result = 9
Operand 1 = 9;  Operand 2 = 2    ->     Multiplied Result = 18
Operand 1 = 9;  Operand 2 = 3    ->     Multiplied Result = 27
Operand 1 = 9;  Operand 2 = 4    ->     Multiplied Result = 36
Operand 1 = 9;  Operand 2 = 5    ->     Multiplied Result = 45
Operand 1 = 9;  Operand 2 = 6    ->     Multiplied Result = 54
Operand 1 = 9;  Operand 2 = 7    ->     Multiplied Result = 63
Operand 1 = 9;  Operand 2 = 8    ->     Multiplied Result = 72
Operand 1 = 9;  Operand 2 = 9    ->     Multiplied Result = 81
Operand 1 = 9;  Operand 2 = 10   ->     Multiplied Result = 90
Operand 1 = 9;  Operand 2 = 11   ->     Multiplied Result = 99
Operand 1 = 9;  Operand 2 = 12   ->     Multiplied Result = 108
Operand 1 = 9;  Operand 2 = 13   ->     Multiplied Result = 117
Operand 1 = 9;  Operand 2 = 14   ->     Multiplied Result = 126
Operand 1 = 9;  Operand 2 = 15   ->     Multiplied Result = 135
Operand 1 = 9;  Operand 2 = 16   ->     Multiplied Result = 144
Operand 1 = 10;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 10;         Operand 2 = 1    ->     Multiplied Result = 10
Operand 1 = 10;         Operand 2 = 2    ->     Multiplied Result = 20
Operand 1 = 10;         Operand 2 = 3    ->     Multiplied Result = 30
Operand 1 = 10;         Operand 2 = 4    ->     Multiplied Result = 40
Operand 1 = 10;         Operand 2 = 5    ->     Multiplied Result = 50
Operand 1 = 10;         Operand 2 = 6    ->     Multiplied Result = 60
Operand 1 = 10;         Operand 2 = 7    ->     Multiplied Result = 70
Operand 1 = 10;         Operand 2 = 8    ->     Multiplied Result = 80
Operand 1 = 10;         Operand 2 = 9    ->     Multiplied Result = 90
Operand 1 = 10;         Operand 2 = 10   ->     Multiplied Result = 100
Operand 1 = 10;         Operand 2 = 11   ->     Multiplied Result = 110
Operand 1 = 10;         Operand 2 = 12   ->     Multiplied Result = 120
Operand 1 = 10;         Operand 2 = 13   ->     Multiplied Result = 130
Operand 1 = 10;         Operand 2 = 14   ->     Multiplied Result = 140
Operand 1 = 10;         Operand 2 = 15   ->     Multiplied Result = 150
Operand 1 = 10;         Operand 2 = 16   ->     Multiplied Result = 160
```

```
                                                    nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749
Operand 1 = 10;         Operand 2 = 16   ->     Multiplied Result = 160
Operand 1 = 11;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 11;         Operand 2 = 1    ->     Multiplied Result = 11
Operand 1 = 11;         Operand 2 = 2    ->     Multiplied Result = 22
Operand 1 = 11;         Operand 2 = 3    ->     Multiplied Result = 33
Operand 1 = 11;         Operand 2 = 4    ->     Multiplied Result = 44
Operand 1 = 11;         Operand 2 = 5    ->     Multiplied Result = 55
Operand 1 = 11;         Operand 2 = 6    ->     Multiplied Result = 66
Operand 1 = 11;         Operand 2 = 7    ->     Multiplied Result = 77
Operand 1 = 11;         Operand 2 = 8    ->     Multiplied Result = 88
Operand 1 = 11;         Operand 2 = 9    ->     Multiplied Result = 99
Operand 1 = 11;         Operand 2 = 10   ->     Multiplied Result = 110
Operand 1 = 11;         Operand 2 = 11   ->     Multiplied Result = 121
Operand 1 = 11;         Operand 2 = 12   ->     Multiplied Result = 132
Operand 1 = 11;         Operand 2 = 13   ->     Multiplied Result = 143
Operand 1 = 11;         Operand 2 = 14   ->     Multiplied Result = 154
Operand 1 = 11;         Operand 2 = 15   ->     Multiplied Result = 165
Operand 1 = 11;         Operand 2 = 16   ->     Multiplied Result = 176
Operand 1 = 12;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 12;         Operand 2 = 1    ->     Multiplied Result = 12
Operand 1 = 12;         Operand 2 = 2    ->     Multiplied Result = 24
Operand 1 = 12;         Operand 2 = 3    ->     Multiplied Result = 36
Operand 1 = 12;         Operand 2 = 4    ->     Multiplied Result = 48
Operand 1 = 12;         Operand 2 = 5    ->     Multiplied Result = 60
Operand 1 = 12;         Operand 2 = 6    ->     Multiplied Result = 72
Operand 1 = 12;         Operand 2 = 7    ->     Multiplied Result = 84
Operand 1 = 12;         Operand 2 = 8    ->     Multiplied Result = 96
Operand 1 = 12;         Operand 2 = 9    ->     Multiplied Result = 108
Operand 1 = 12;         Operand 2 = 10   ->     Multiplied Result = 120
Operand 1 = 12;         Operand 2 = 11   ->     Multiplied Result = 132
Operand 1 = 12;         Operand 2 = 12   ->     Multiplied Result = 144
Operand 1 = 12;         Operand 2 = 13   ->     Multiplied Result = 156
Operand 1 = 12;         Operand 2 = 14   ->     Multiplied Result = 168
Operand 1 = 12;         Operand 2 = 15   ->     Multiplied Result = 180
Operand 1 = 12;         Operand 2 = 16   ->     Multiplied Result = 192
Operand 1 = 13;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 13;         Operand 2 = 1    ->     Multiplied Result = 13
Operand 1 = 13;         Operand 2 = 2    ->     Multiplied Result = 26
Operand 1 = 13;         Operand 2 = 3    ->     Multiplied Result = 39
Operand 1 = 13;         Operand 2 = 4    ->     Multiplied Result = 52
Operand 1 = 13;         Operand 2 = 5    ->     Multiplied Result = 65
Operand 1 = 13;         Operand 2 = 6    ->     Multiplied Result = 78
Operand 1 = 13;         Operand 2 = 7    ->     Multiplied Result = 91
Operand 1 = 13;         Operand 2 = 8    ->     Multiplied Result = 104
Operand 1 = 13;         Operand 2 = 9    ->     Multiplied Result = 117
Operand 1 = 13;         Operand 2 = 10   ->     Multiplied Result = 130
Operand 1 = 13;         Operand 2 = 11   ->     Multiplied Result = 143
Operand 1 = 13;         Operand 2 = 12   ->     Multiplied Result = 156
Operand 1 = 13;         Operand 2 = 13   ->     Multiplied Result = 169
Operand 1 = 13;         Operand 2 = 14   ->     Multiplied Result = 182
Operand 1 = 13;         Operand 2 = 15   ->     Multiplied Result = 195
Operand 1 = 13;         Operand 2 = 16   ->     Multiplied Result = 208
Operand 1 = 14;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 14;         Operand 2 = 1    ->     Multiplied Result = 14
Operand 1 = 14;         Operand 2 = 2    ->     Multiplied Result = 28
Operand 1 = 14;         Operand 2 = 3    ->     Multiplied Result = 42
Operand 1 = 14;         Operand 2 = 4    ->     Multiplied Result = 56
Operand 1 = 14;         Operand 2 = 5    ->     Multiplied Result = 70
Operand 1 = 14;         Operand 2 = 6    ->     Multiplied Result = 84
Operand 1 = 14;         Operand 2 = 7    ->     Multiplied Result = 98
Operand 1 = 14;         Operand 2 = 8    ->     Multiplied Result = 112
Operand 1 = 14;         Operand 2 = 9    ->     Multiplied Result = 126
Operand 1 = 14;         Operand 2 = 10   ->     Multiplied Result = 140
Operand 1 = 14;         Operand 2 = 11   ->     Multiplied Result = 154
Operand 1 = 14;         Operand 2 = 12   ->     Multiplied Result = 168
Operand 1 = 14;         Operand 2 = 13   ->     Multiplied Result = 182
Operand 1 = 14;         Operand 2 = 14   ->     Multiplied Result = 196
Operand 1 = 14;         Operand 2 = 15   ->     Multiplied Result = 210
Operand 1 = 14;         Operand 2 = 16   ->     Multiplied Result = 224
Operand 1 = 15;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 15;         Operand 2 = 1    ->     Multiplied Result = 15
Operand 1 = 15;         Operand 2 = 2    ->     Multiplied Result = 30
Operand 1 = 15;         Operand 2 = 3    ->     Multiplied Result = 45
```

```
Operand 1 = 15;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 15;         Operand 2 = 1    ->     Multiplied Result = 15
Operand 1 = 15;         Operand 2 = 2    ->     Multiplied Result = 30
Operand 1 = 15;         Operand 2 = 3    ->     Multiplied Result = 45
Operand 1 = 15;         Operand 2 = 4    ->     Multiplied Result = 60
Operand 1 = 15;         Operand 2 = 5    ->     Multiplied Result = 75
Operand 1 = 15;         Operand 2 = 6    ->     Multiplied Result = 90
Operand 1 = 15;         Operand 2 = 7    ->     Multiplied Result = 105
Operand 1 = 15;         Operand 2 = 8    ->     Multiplied Result = 120
Operand 1 = 15;         Operand 2 = 9    ->     Multiplied Result = 135
Operand 1 = 15;         Operand 2 = 10   ->     Multiplied Result = 150
Operand 1 = 15;         Operand 2 = 11   ->     Multiplied Result = 165
Operand 1 = 15;         Operand 2 = 12   ->     Multiplied Result = 180
Operand 1 = 15;         Operand 2 = 13   ->     Multiplied Result = 195
Operand 1 = 15;         Operand 2 = 14   ->     Multiplied Result = 210
Operand 1 = 15;         Operand 2 = 15   ->     Multiplied Result = 225
Operand 1 = 15;         Operand 2 = 16   ->     Multiplied Result = 240
Operand 1 = 16;         Operand 2 = 0    ->     Multiplied Result = 0
Operand 1 = 16;         Operand 2 = 1    ->     Multiplied Result = 16
Operand 1 = 16;         Operand 2 = 2    ->     Multiplied Result = 32
Operand 1 = 16;         Operand 2 = 3    ->     Multiplied Result = 48
Operand 1 = 16;         Operand 2 = 4    ->     Multiplied Result = 64
Operand 1 = 16;         Operand 2 = 5    ->     Multiplied Result = 80
Operand 1 = 16;         Operand 2 = 6    ->     Multiplied Result = 96
Operand 1 = 16;         Operand 2 = 7    ->     Multiplied Result = 112
Operand 1 = 16;         Operand 2 = 8    ->     Multiplied Result = 128
Operand 1 = 16;         Operand 2 = 9    ->     Multiplied Result = 144
Operand 1 = 16;         Operand 2 = 10   ->     Multiplied Result = 160
Operand 1 = 16;         Operand 2 = 11   ->     Multiplied Result = 176
Operand 1 = 16;         Operand 2 = 12   ->     Multiplied Result = 192
Operand 1 = 16;         Operand 2 = 13   ->     Multiplied Result = 208
Operand 1 = 16;         Operand 2 = 14   ->     Multiplied Result = 224
Operand 1 = 16;         Operand 2 = 15   ->     Multiplied Result = 240
Operand 1 = 16;         Operand 2 = 16   ->     Multiplied Result = 256

*** local echo: yes ***

*** local echo: no ***

*** local echo: yes ***

Terminating...
Thanks for using picocom
nithingowtham25@zach-354a-02:~/Spring_2026/ECEN749$
```